Advanced Procedures Chapter-8

Stack Frame

A Stack frame is the area of the stack set aside for a procedure's return address, passed parameters, saved registers, and local variables.
Also known as an **Activation Record**.

Created by the following steps:

- Passed arguments, if any, are pushed on the stack.
- The subroutine is called, causing the subroutine return address to be pushed on the stack.
- As the subroutine begins to execute, EBP is pushed on the stack.

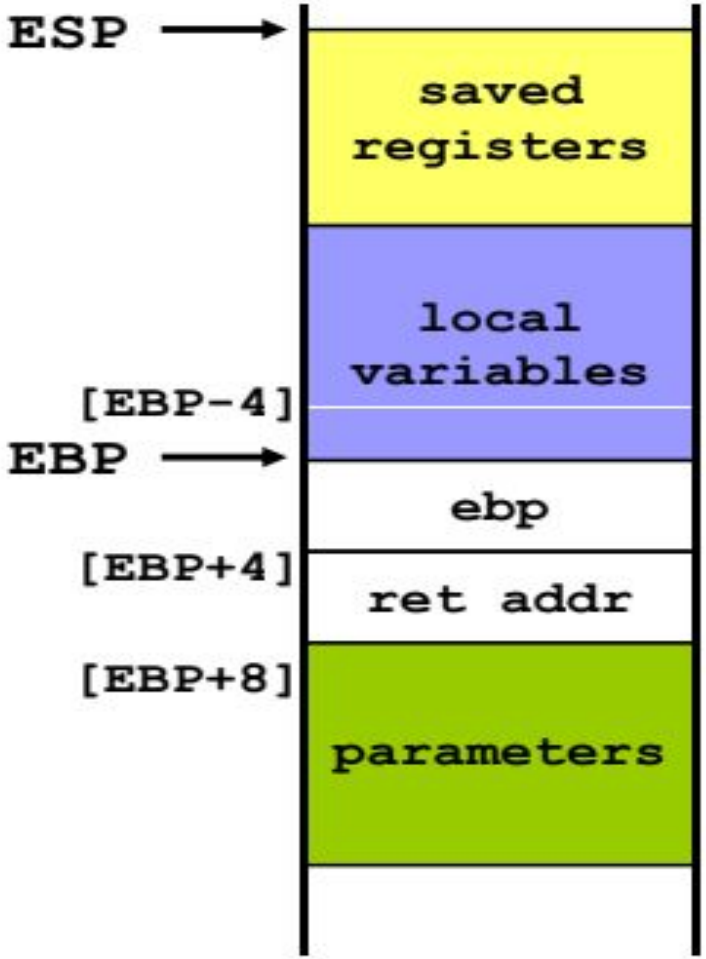
Stack Frame

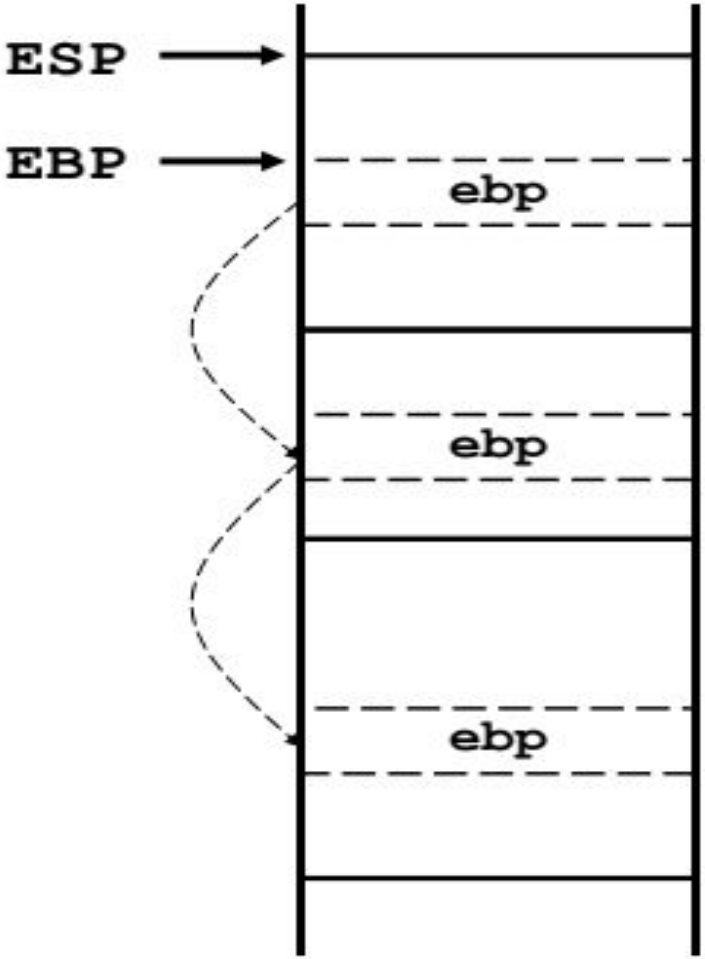
- EBP is set equal to ESP. From this point on, EBP acts as a base reference for all of the subroutine parameters.

- If there are local variables, ESP is decremented to reserve space for the variables on the stack.

- If any registers need to be saved, they are pushed on the stack.

The structure of a stack frame is directly affected by a program's memory model and its choice of argument passing convention.


ESP
saved registers
local variables
[EBP-4]
EBP
ebp
[EBP+4]
ret addr
[EBP+8]
parameters


ESP
EBP
ebp
ebp
ebp

DISADVANTAGES OF REGISTER PARAMETERS

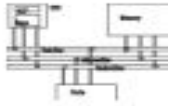
- In earlier chapters, our subroutines received register parameters.
- Because registers' use is multipurpose, the registers used as parameters must:
  -
  - first be pushed on the stack before procedure calls,
  - assigned the values of procedure arguments,
  - and later restored to their original values after the procedure returns.

•Extra Pushes/Pops

•Programmers have to be very careful that each register's PUSH is matched by its appropriate POP.

```
push ebx                              ; save register values
push ecx
push esi

mov esi, OFFSET array                 ; use registers as parameters
mov ecx, LENGTHOF array
mov ebx, TYPE array
call DumpMem

pop esi                               ; restore register values
pop ecx
pop ebx
```

Explicit access to stack parameters



- A procedure can explicitly access stack parameters using constant offsets from **EBP**.
  - Example: `[ebp + 8]`
- **EBP** is often called the base pointer or frame pointer because it holds the base address of the stack frame.
- **EBP** does not change value during the procedure.
- **EBP** must be restored to its original value when a procedure returns.

- Stack parameters offer a flexible approach that does not require register parameters. Just before the subroutine call, the arguments are pushed on the stack.

```
push TYPE array
push LENGTHOF array
push OFFSET array
call DumpMem
```

- Two general types of arguments are pushed on the stack during subroutine calls:

1. Value arguments (values of variables and constants)
2. Reference arguments (addresses of variables)

Parameters

call by value

```
int sum=AddTwo(a, b);
```

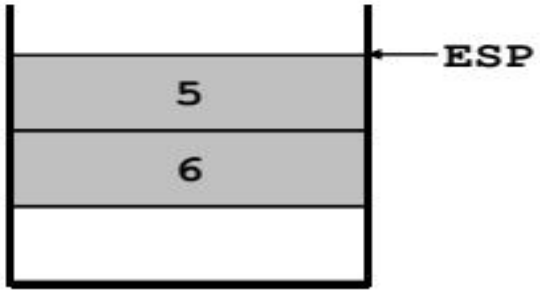
call by reference

```
int sum=AddTwo(&a, &b);
```

```
.date
a   DWORD   5
b   DWORD   6
```

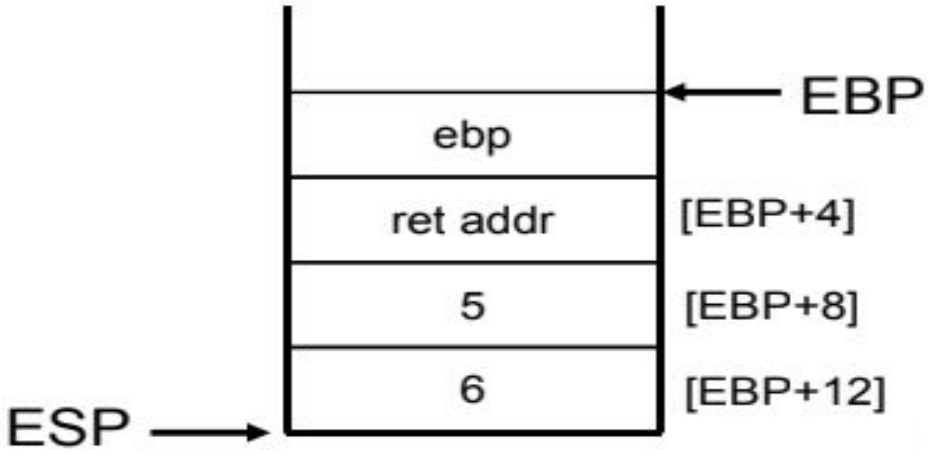
```
push b
push a
call AddTwo
```

```
push OFFSET b
push OFFSET a
call AddTwo
```



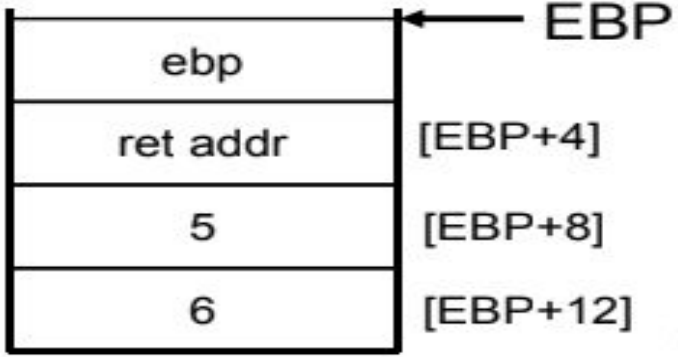
```
.data
sum DWORD ?
.code
   push 6                ; second argument
   push 5                ; first argument
   call AddTwo           ; EAX = sum
   mov  sum,eax          ; save the sum
```

```
AddTwo PROC
   push ebp
   mov  ebp,esp
   .
   .
```

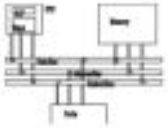


```
AddTwo PROC
   push ebp
   mov ebp,esp              ; base of stack frame
   mov eax,[ebp + 12]       ; second argument (6)
   add eax,[ebp + 8]        ; first argument (5)
   pop ebp
   ret 8                    ; clean up the stack
AddTwo ENDP                 ; EAX contains the sum
```

Who should be responsible to remove arguments? It depends on the language model.



`RET` Instruction



- ***Return from subroutine***
- Pops stack into the instruction pointer (EIP or IP). Control transfers to the target address.
- Syntax:
  - **`RET`**
  - **`RET`** ***n***
- Optional operand ***n*** causes ***n*** bytes to be added to the stack pointer after EIP (or IP) is assigned a value.

PASSING ARRAY

- High-level languages always pass arrays to subroutines by reference. That is, they push the address of an array on the stack. The subroutine can then get the address from the stack and use it to access the array.

```
.data
        array DWORD 50 DUP(?)
.code
        push OFFSET array
        call ArrayFill
```

Accessing Stack Parameter

- EBP is called the **base pointer** or **frame pointer** because it holds the base address of the stack frame.
- Given the following C++ function:

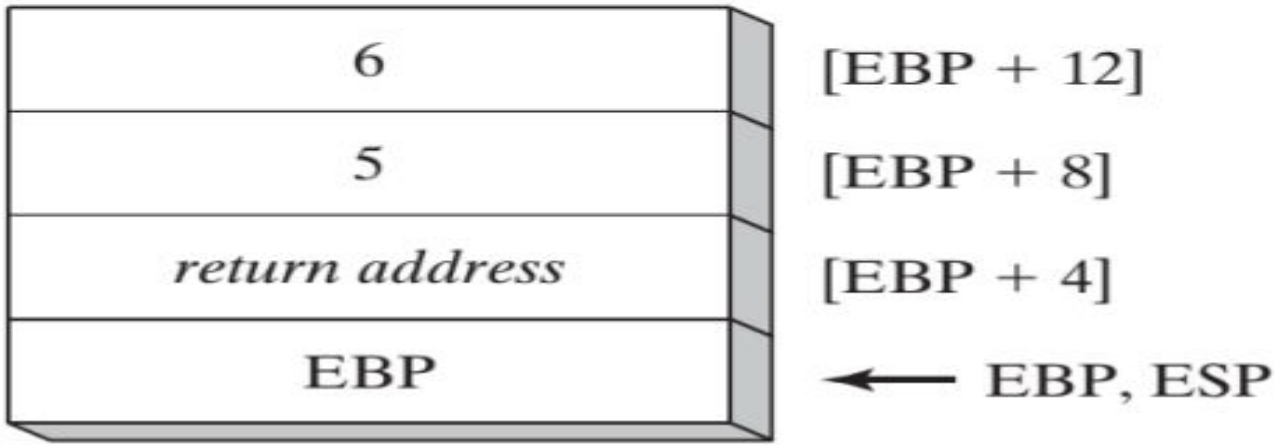
```
int AddTwo( int x, int y )
{
        return x + y;
}
```

- A function call such as **`AddTwo(5,6)`** would cause the second parameter to be pushed on the stack, followed by the first parameter.

- **The Assembly Equivalent:** In its earliest execution, **AddTwo** pushes EBP on the stack to preserve its existing value, and then EBP is set to the same value as ESP, so EBP can be the base pointer for AddTwo's stack frame:

```
AddTwo PROC
       push ebp
       mov ebp,esp
```

- ESP would change value, but EBP would not.
- Base-Offset Addressing is used to access stack parameters.
  - EBP is the base register and the offset is a constant (***explicit stack parameters)*** .



```
AddTwo PROC
       push ebp
       mov ebp,esp             ; base of stack frame
       mov eax,[ebp + 12]      ; second parameter
       add eax,[ebp + 8]       ; first parameter
       pop ebp
       ret
AddTwo ENDP
```

- When stack parameters are referenced with expressions such as `[ebp + 8]`, we call them ***explicit stack parameters.***

Explicit Stack Parameters

When stack parameters are referenced with expressions such as [ebp + 8], we call them *explicit stack parameters*. The reason for this term is that the assembly code explicitly states the offset of the parameter as a constant value. Some programmers define symbolic constants to represent the explicit stack parameters, to make their code easier to read:

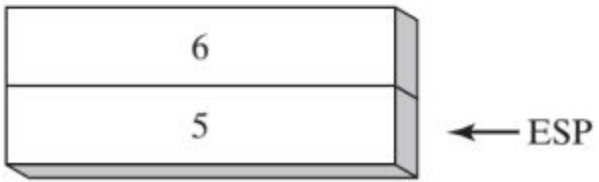
```
y_param EQU [ebp + 12]
x_param EQU [ebp + 8]

AddTwo PROC
     push  ebp

 mov   ebp,esp
 mov   eax,y_param
 add   eax,x_param
 pop   ebp
 ret
AddTwo ENDP
```

Cleaning Up the Stack

- There must be a way for parameters to be removed from the stack when a subroutine returns.

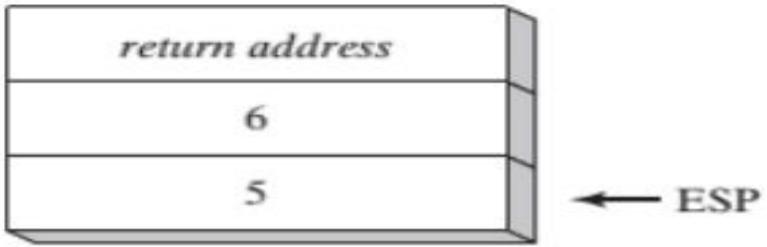
```
push 6
push 5
call AddTwo
```

Assuming that `AddTwo` leaves the two parameters on the stack, the following illustration shows the stack after returning from the call:



```
main PROC
 call Example1
 exit
main ENDP

Example1 PROC
  push 6
  push 5
  call AddTwo
  ret                              ; stack is corrupted!
Example1 ENDP
```

- When the RET instruction in Example1 is about to execute, ESP points to the integer 5 rather than the return address that would take it back to main:



- if we were to call AddTwo from a loop, the stack could overflow. Each call uses 12 bytes of stack space:
  - 4 bytes for each parameter, plus 4 bytes for the CALL instruction's return address.

- **32-Bit Calling Conventions**

1. **The C Calling Convention:** The C calling convention is used by the C and C++ programming languages.
   - Subroutine parameters are pushed on the stack in reverse order.

- The C calling convention solves the problem of cleaning up the runtime stack in a simple way:
- When a program calls a subroutine, it follows the CALL instruction with a statement that adds a value to the stack pointer (ESP) equal to the combined sizes of the subroutine parameters.

```
Example1 PROC
        push 6
        push 5
        call AddTwo
        add esp,8          ; remove arguments from the stack
        ret
Example1 ENDP
```

2. **STDCALL Calling Convention:** Another common way to remove parameters from the stack is to use a convention named STDCALL.

- We supply an integer parameter to the RET instruction, which in turn adds integer to ESP after returning to the calling procedure.
- The integer must equal the number of bytes of stack space consumed by the procedure's parameters

```
AddTwo PROC
        push ebp
        mov ebp,esp                 ; base of stack frame
        mov eax,[ebp + 12]          ; second parameter
        add eax,[ebp + 8]           ; first parameter
        pop ebp
        ret 8                       ; clean up the stack
AddTwo ENDP
```

SAVING AND RESTORING REGISTERS

- Subroutines often save the current contents of registers on the stack before modifying them so that the original values can be restored just before the subroutine returns.

```
MySub PROC
        push ebp                    ; save base pointer
        mov ebp,esp                 ; base of stack frame
        push ecx
        push edx                    ; save EDX
        mov eax,[ebp+8]             ; get the stack parameter

        pop edx                     ; restore saved registers
        pop ecx
        pop ebp                     ; restore base pointer
        ret                         ; clean up the stack
MySub ENDP
```

LOCAL VARIABLES

- Local variables are created on the runtime stack, usually below the base pointer (EBP).

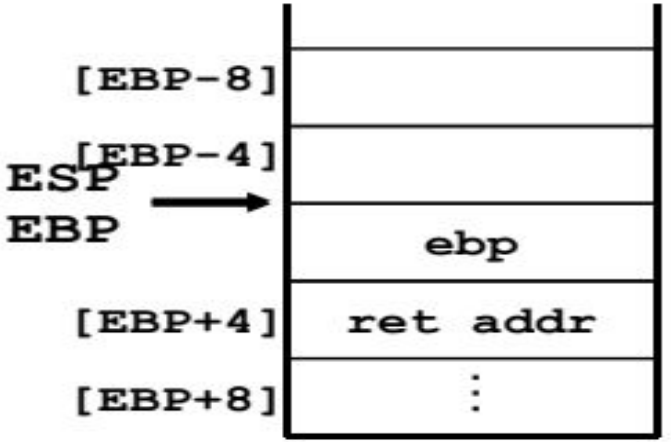
```
void MySub()
{
        int X = 10;
        int Y = 20;
}
```

Variable	Bytes	Stack Offset
X	4	EBP − 4
Y	4	EBP − 8

Creating local variables

- Local variables are created on the runtime stack, usually above EBP.
- To explicitly create local variables, subtract their total size from ESP.

```
MySub PROC
   push ebp
   mov  ebp,esp
   sub  esp,8
   mov  [ebp-4],123456h
   mov  [ebp-8],0
   .
   .
```

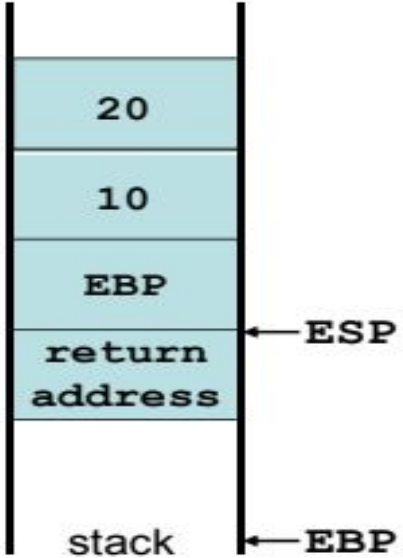


Local variables

- They can't be initialized at assembly time but can be assigned to default values at runtime.

```
void MySub()
{
  int X=10;
  int Y=20;
  ...
}
```

```
MySub PROC
   push ebp
   mov  ebp, esp
   sub  esp, 8
   mov  DWORD PTR [ebp-4], 10
   mov  DWORD PTR [ebp-8], 20
   ...
   mov  esp, ebp
   pop  ebp
   ret
MySub ENDP
```



Local Variable Symbols

```
X_local EQU DWORD PTR [ebp-4]
Y_local EQU DWORD PTR [ebp-8]

MySub PROC
   push ebp
   mov  ebp, esp
   sub  esp, 8
   mov  X_local, 10
   mov  Y_local, 20
   ...
   mov  esp, ebp
   pop  ebp
   ret
MySub ENDP
```

LEA Instruction

- LEA returns offsets of direct and indirect operands
  - OFFSET operator only returns constant offsets
- LEA required when obtaining offsets of stack parameters & local variables
- Example

```
CopyString PROC,
    count:DWORD
    LOCAL temp[20]:BYTE

    mov edi,OFFSET count        ; invalid operand
    mov esi,OFFSET temp         ; invalid operand
    lea edi,count               ; ok
    lea esi,temp                ; ok
```

LEA Example

Suppose you have a Local variable at [ebp-8]

And you need the address of that local variable in ESI

You cannot use this:

```
mov esi, OFFSET [ebp-8]          ; error
```

Use this instead:

```
lea esi,[ebp-8]
```

LEA INSTRUCTION

- The LEA instruction returns the address of an indirect operand.
- Ideally suited for use with stack parameters

```
void makeArray()
{
char myString[30];
for( int i = 0; i 30; i++
)
    myString[i] = '*';
}
```

```
makeArray PROC
 push ebp
 mov ebp,esp
 sub esp,32
 lea esi,[ebp-30]
 mov ecx,30
 L1: mov BYTE PTR [esi],'*'
 inc esi
 loop L1
 add esp,32 ; (restore ESP)
 pop ebp
 ret
makeArray ENDP
```

ENTER AND LEAVE INSTRUCTIONS

- The **ENTER** instruction performs three operations:

1. Pushes `EBP` on the stack (`push ebp`)
2. Sets `EBP` to the base of the stack frame (`mov ebp, esp`)
3. Reserves space for local variables (`sub esp,numbytes`)

ENTER *numbytes, nestinglevel*

- Both the operands are immediate values,
- The first is a constant specifying the number of bytes of stack space to reserve for local variables
- The second specifies the lexical nesting level of the procedure.

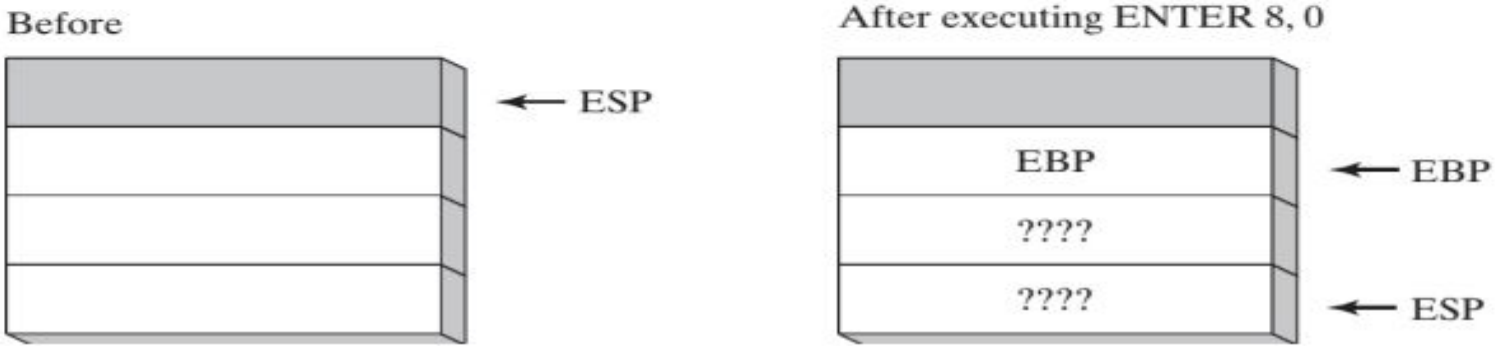
E.g. a procedure with no local variables:

```
MySub PROC
    ENTER 0,0
```

If the nesting level is 0, the processor pushes the frame pointer from the BP/EBP/RBP register onto the stack, copies the current stack pointer from the SP/ESP/RSP register into the BP/EBP/RBP register, and loads the SP/ESP/RSP register with the current stack-pointer value minus the value in the size operand. For nesting levels of 1 or greater, the processor pushes additional frame pointers on the stack before adjusting the stack pointer. These additional frame pointers provide the called procedure with access points to other nested frames on the stack.

E.g. The ENTER instruction reserves 8 bytes of stack space for local variables

```
MySub PROC
    ENTER 8,0
```



- The **LEAVE** instruction terminates the stack frame for a procedure.
  - It reverses the action of a previous ENTER instruction by restoring ESP and EBP to the values they were assigned when the procedure was called.

```
MySub PROC
        enter 8,0
        .
        .
        leave
        ret
MySub ENDP
```

LOCAL DIRECTIVE

LOCAL declares one or more local variables by name, assigning them size attributes.

ENTER, on the other hand, only reserves a single unnamed block of stack space for local variables.

If used, LOCAL must appear on the line immediately following the PROC directive.

```
MySub PROC
      LOCAL var1:BYTE
```

LOCAL directive

- The **LOCAL** directive declares a list of local variables
  - immediately follows the **PROC** directive
  - each variable is assigned a type
- Syntax:

  LOCAL ***varlist***

Example:

```
MySub PROC
   LOCAL var1:BYTE, var2:WORD, var3:SDWORD
```

MASM-generated code

```
BubbleSort PROC
   LOCAL temp:DWORD, SwapFlag:BYTE
   . . .
   ret
BubbleSort ENDP
```

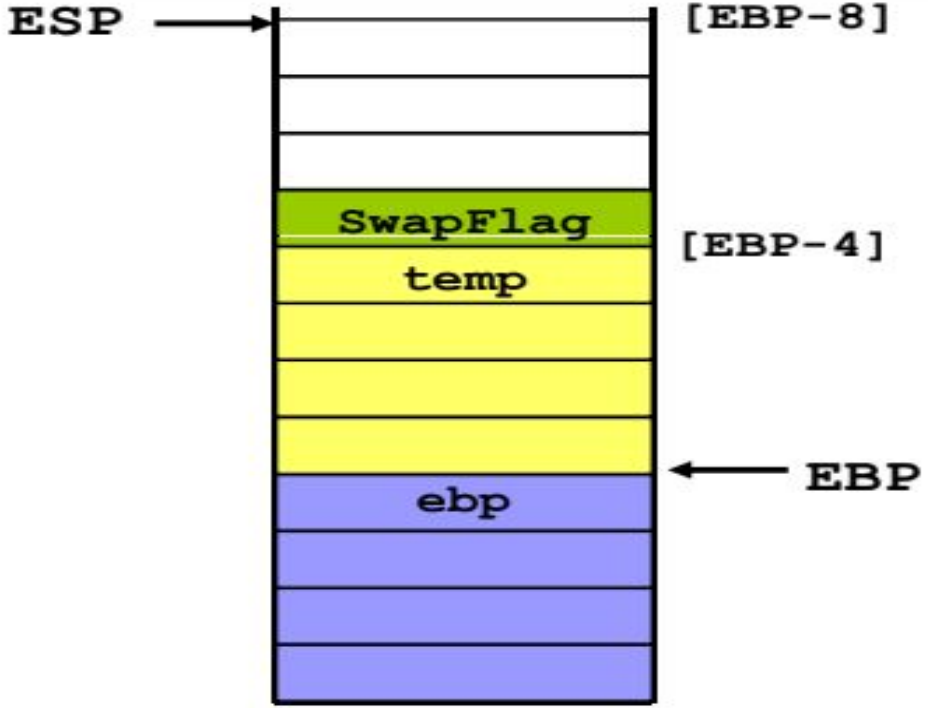
MASM generates the following code:

```
BubbleSort PROC
   push ebp
   mov  ebp,esp
   add  esp,0FFFFFFF8h ; add -8 to ESP
   . . .
   mov  esp,ebp
   pop  ebp
   ret
BubbleSort ENDP
```

Non-Doubleword Local Variables

- Local variables can be different sizes
- How are they created in the stack by **`LOCAL`** directive:
  - 8-bit: assigned to next available byte
  - 16-bit: assigned to next even (word) boundary
  - 32-bit: assigned to next doubleword boundary

MASM-generated code




```
mov   eax, temp          mov   eax, [ebp-4]
mov   bl, SwapFlag  ---> mov   bl, [ebp-5]
```

Reserving stack space

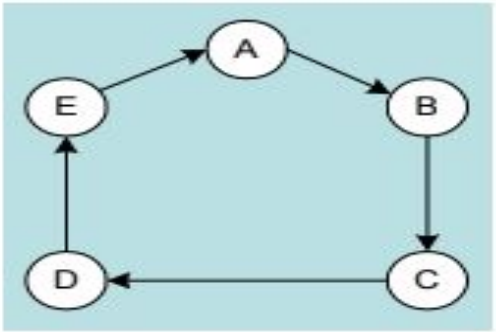
- `.STACK 4096`
- **Sub1** calls **Sub2**, **Sub2** calls **Sub3**, how many bytes will you need in the stack?

```
Sub1 PROC
  LOCAL array1[50]:DWORD ; 200 bytes

Sub2 PROC
  LOCAL array2[80]:WORD  ; 160 bytes

Sub3 PROC
  LOCAL array3[300]:WORD ; 300 bytes
```

660+8(ret addr)+saved registers...

Recursion

- The process created when . . .
  - A procedure calls itself
  - Procedure A calls procedure B, which in turn calls procedure A
- Using a graph in which each node is a procedure and each edge is a procedure call, recursion forms a cycle:

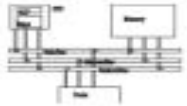


```
INCLUDE Irvine32.inc

.data
        endlessStr BYTE "This recursion never stops",0

.code
  main PROC
    call Endless
    exit
  main ENDP

  Endless PROC
    mov edx,OFFSET endlessStr
    call WriteString
    call Endless
    ret                                  ; never executes
  Endless ENDP

END main
```

```
INCLUDE Irvine32.inc
.code
main PROC
        mov ecx,5                       ; count = 5
        mov eax,0                       ; holds the sum
        call CalcSum                    ; calculate sum
        L1: call WriteDec               ; display EAX
        call Crlf                       ; new line
        exit
main ENDP

CalcSum PROC
        cmp ecx,0                       ; check counter value
        jz L2                           ; quit if zero
        add eax,ecx                     ; otherwise, add to sum
        dec ecx                         ; decrement counter
        call CalcSum                    ; recursive call
        L2: ret
CalcSum ENDP
end Main
```

Pushed on Stack	Value in ECX	Value in EAX
L1	5	0
L2	4	5
L2	3	9
L2	2	12
L2	1	14
L2	0	15

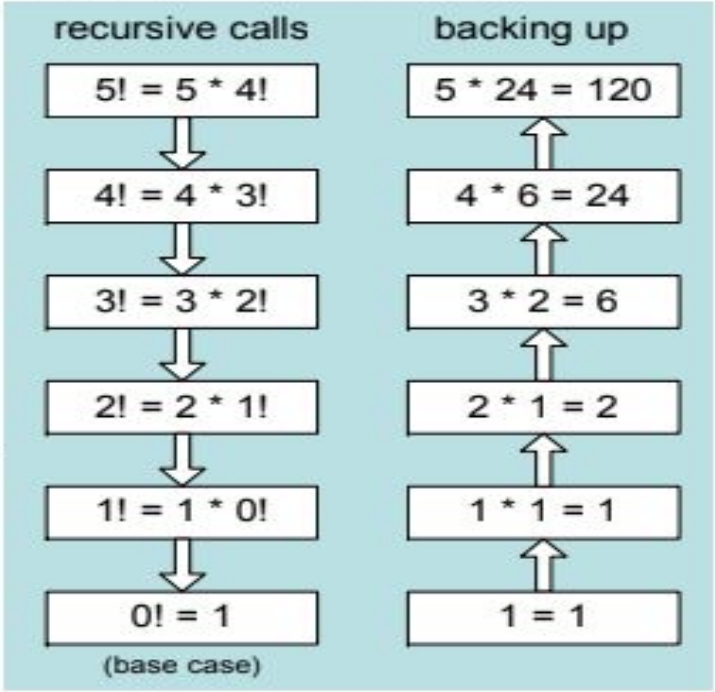
Calculating a factorial



This function calculates the factorial of integer ***n***.
A new value of ***n*** is saved in each stack frame:

```
int factorial(int n)
{
  if (n == 0)
    return 1;
  else
    return n*factorial(n-1);
}
```

```
factorial(5);
```

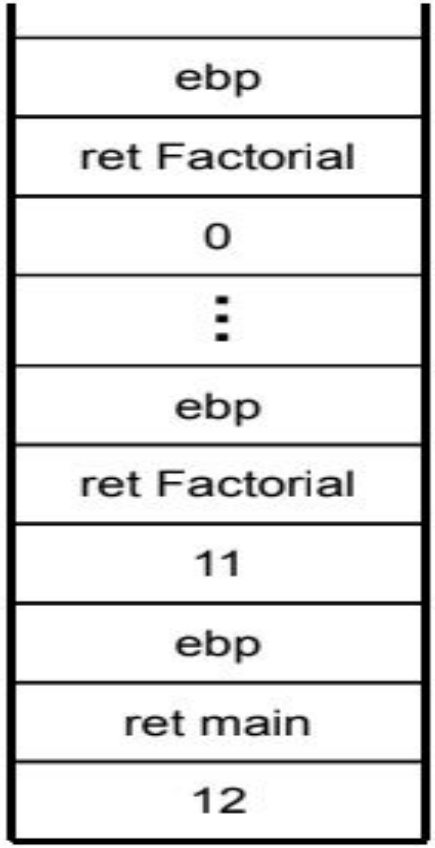


```
Factorial PROC
    push ebp
    mov  ebp,esp
    mov  eax,[ebp+8]      ; get n
    cmp  eax,0            ; n > 0?
    ja   L1               ; yes: continue
    mov  eax,1            ; no: return 1
    jmp  L2
L1:dec  eax
    push eax              ; Factorial(n-1)
    call Factorial

ReturnFact:
    mov  ebx,[ebp+8]        ; get n
    mul  ebx                ; edx:eax=eax*ebx

L2:pop  ebp               ; return EAX
    ret  4                ; clean up stack
Factorial ENDP
```

Calculating a factorial

```
push 12
call Factorial
```

```
Factorial PROC
    push ebp
    mov  ebp,esp
    mov  eax,[ebp+8]
    cmp  eax,0
    ja   L1
    mov  eax,1
    jmp  L2
L1:dec  eax
    push eax
    call Factorial

ReturnFact:
    mov  ebx,[ebp+8]
    mul  ebx

L2:pop  ebp
    ret  4
Factorial ENDP
```



INVOKE , ADDR, PROC, PROTO

- In 32-bit mode, the `INVOKE`, `PROC`, and `PROTO` directives provide powerful tools for defining and calling procedures.

- The `ADDR` operator is an essential tool for defining procedure parameters.

- Their use is controversial because they mask the underlying structure of the runtime stack.

INVOKE DIRECTIVE

- The `INVOKE` directive, only available in 32-bit mode, pushes arguments on the stack and calls a procedure.

- `INVOKE` is a convenient replacement for the `CALL` instruction because it lets you pass multiple arguments using a single line of code.

INVOKE *procedureName [, argumentList]*

CALL VS INVOKE

```
push TYPE array
push LENGTHOF array
push OFFSET array
call DumpArray
```

The equivalent statement using INVOKE is reduced to a single line in which the arguments are listed in reverse order (assuming STDCALL is in effect)

```
INVOKE DumpArray, OFFSET array, LENGTHOF array, TYPE array
```

INVOKE permits almost any number of arguments, and individual arguments can appear on separate source code lines

Type	Examples
Immediate value	10, 3000h, OFFSET mylist, TYPE array
Integer expression	(10 * 20), COUNT
Variable	myList, array, myWord, myDword
Address expression	[myList+2], [ebx + esi]
Register	eax, bl, edi
ADDR *name*	ADDR myList
OFFSET *name*	OFFSET myList

```
.data
    byteVal BYTE 10
    wordVal WORD 1000h
.code
    ; direct operands:
    INVOKE Sub1,byteVal,wordVal

    ; address of variable:
    INVOKE Sub2,ADDR byteVal

    ; register name, integer expression:
    INVOKE Sub3,eax,(10 * 20)

    ; address expression (indirect operand):
    INVOKE Sub4,[ebx]
```

ADDR OPERATOR

- The ADDR operator, also available in 32-bit mode, can be used to pass a pointer argument when calling a procedure using INVOKE:

```
INVOKE FillArray, ADDR myArray
```

- The ADDR operator can only be used in conjunction with INVOKE:

```
mov esi, ADDR myArray                    ; error
```

- The argument passed to ADDR must be an assembly time constant.

```
INVOKE mySub, ADDR [ebp+12]              ; error
```

ADDR Operator

- Returns a near or far pointer to a variable, depending on which memory model your program uses:
  - *Small model*: returns 16-bit offset
  - *Large model*: returns 32-bit segment/offset
  - *Flat model*: returns 32-bit offset
- Simple example:

```
.data
myWord WORD ?
.code
INVOKE mySub,ADDR myWord
```

PROC DIRECTIVE

- The PROC directive declares a procedure with an optional list of named parameters.

label **PROC**, *parameter_list*

- The PROC directive permits you to declare a procedure with a comma-separated list of named parameters.

```
label PROC, parameter_1, parameter_2, ..., parameter_n
```

- Your implementation code can refer to the parameters by name rather than by calculated stack offsets such as [ebp - 8].

- A single parameter has the following syntax:

```
paramName: type
```

- *ParamName* is an arbitrary name you assign to the parameter. Its scope is limited to the current procedure (called *local scope*).

- *Type* can be one of : BYTE, SBYTE, WORD, SWORD, DWORD, SDWORD, FWORD, QWORD, or TBYTE may be a pointer to an existing type (qualified type)

```
PTR BYTE                PTR SBYTE
PTR WORD                PTR SWORD
PTR DWORD               PTR SDWORD
PTR QWORD               PTR TBYTE
```

```
AddTwo PROC,
 val1:DWORD,
 val2:DWORD

 mov eax,val1
 add eax,val2
 ret
AddTwo ENDP
```

```
AddTwo PROC,
 push ebp
 mov  ebp, esp
 mov  eax, dword ptr [ebp+8]
 add  eax, dword ptr [ebp+0Ch]
 leave
 ret 8
AddTwo ENDP
```

```
FillArray PROC,
    pArray:PTR BYTE,
    fillVal:BYTE,
    arraySize:DWORD

    mov ecx,arraySize
    mov esi,pArray
    mov al,fillVal

L1: mov [esi],al
    inc esi
    loop L1
    ret
FillArray ENDP
```

Example 1 The AddTwo procedure receives two doubleword values and returns their sum in EAX:

```
AddTwo PROC,
     val1:DWORD,
     val2:DWORD
     mov   eax,val1
     add   eax,val2
     ret
AddTwo ENDP
```

Example 2 The FillArray procedure receives a pointer to an array of bytes:

```
FillArray PROC,
     pArray:PTR BYTE
     . . .
FillArray ENDP
```

Example 3 The Swap procedure receives two pointers to doublewords:

```
Swap PROC,
     pValX:PTR DWORD,
     pValY:PTR DWORD
     . . .
Swap ENDP
```

USES OPERATOR

- The USES operator, coupled with the PROC directive, lets you list the names of all registers modified within a procedure.
- USES tells the assembler to do two things:

1. First, generate PUSH instructions that save the registers on the stack at the beginning of the procedure.
2. Second, generate POP instructions that restore the register values at the end of the procedure.

```
ArraySum PROC USES esi ecx
     mov   eax,0
L1:
     add   eax,[esi]
     add   esi,TYPE DWORD
     loop  L1

     ret
ArraySum ENDP
```

```
ArraySum PROC
     push  esi
     push  ecx
     mov   eax,0
L1:
     add   eax,[esi]
     add   esi,TYPE DWORD
     loop  L1

     pop   ecx
     pop   esi
     ret
ArraySum ENDP
```

PROTO DIRECTIVE

- Creates a procedure prototype

label PROTO paramList

- Every procedure called by the INVOKE directive must have a prototype.

- A complete procedure definition can also serve as its own prototype.

- Standard configuration: PROTO appears at top of the program listing, INVOKE appears in the code segment, and the procedure implementation occurs later in the program.

```
MySub PROTO          ; procedure prototype

.code
   INVOKE MySub      ; procedure call


MySub PROC           ; procedure implementation
    .
    .
MySub ENDP
```

Prototype for the ArraySum procedure, showing its parameter list:

```
ArraySum PROTO,
    ptrArray:PTR DWORD,        ; points to the array
    szArray:DWORD              ; array size
```

DEBUGGING TIPS

ARGUMENT SIZE MISMATCH

- Array addresses are based on the sizes of their elements.
  - To address the second element of a doubleword array, for example, one adds 4 to the array's starting address.

- Suppose we call a procedure passing pointers to the first two elements of **DoubleArray**. If we incorrectly calculate the address of the second element as **DoubleArray + 1**, the resulting values after calling that procedure are incorrect:

PASSING IMMEDIATE VALUES

- If a procedure has a reference parameter, do not pass it an immediate argument.

```
Sub2 PROC, dataPtr:PTR WORD
        mov esi,dataPtr         ; get the address
        mov WORD PTR [esi],0    ; dereference, assign zero
        ret
Sub2 ENDP
```

- The following INVOKE statement assembles but causes a runtime error. The **Sub2** procedure receives 1000h as a pointer value and dereferences memory location 1000h:

```
INVOKE Sub2, 1000h
```

PASSING 64-BIT ARGUMENTS

- In 32-bit mode, when passing 64-bit integer arguments to subroutines on the stack, push the high-order doubleword of the argument first, followed by the low-order doubleword.
- Doing so places the integer into the stack in little-endian order (low order byte at the lowest address)

```
.data
longVal QWORD 1234567800ABCDEFh
.code
push  DWORD PTR longVal + 4     ; high doubleword
push  DWORD PTR longVal         ; low doubleword
call  WriteHex64
```

Stack frame after pushing EBP.

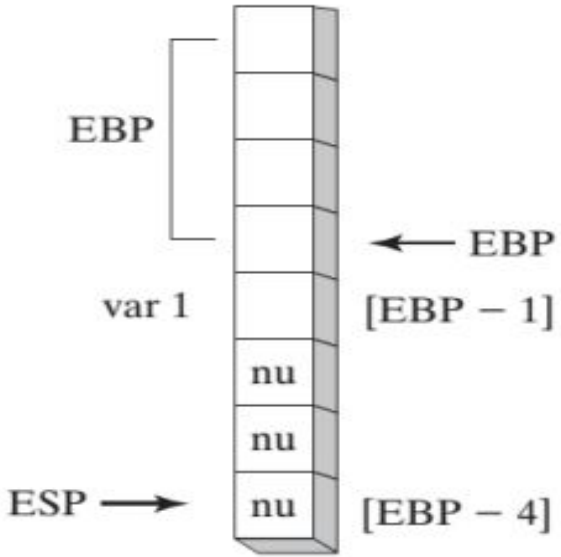

```
WriteHex64 PROC
     push  ebp
     mov   ebp,esp
     mov   eax,[ebp+12]          ; high doubleword
     call  WriteHex
     mov   eax,[ebp+8]           ; low doubleword
     call  WriteHex
     pop   ebp
     ret   8
WriteHex64 ENDP
```

NON-DOUBLEWORD LOCAL VARIABLES

- The LOCAL directive has interesting behavior when you declare local variables of differing sizes. Each is allocated space according to its size
  - An 8-bit variable is assigned to the next available byte, a 16-bit variable is assigned to the next even address (word-aligned), and a 32-bit variable is allocated the next doubleword aligned boundary.

```
Example1 PROC
    LOCAL var1:byte
    mov  al,var1                ; [EBP - 1]
    ret
Example1 ENDP
```

Because stack offsets default to 32 bits in 32-bit mode, one might expect var1 to be located at EBP – 4.



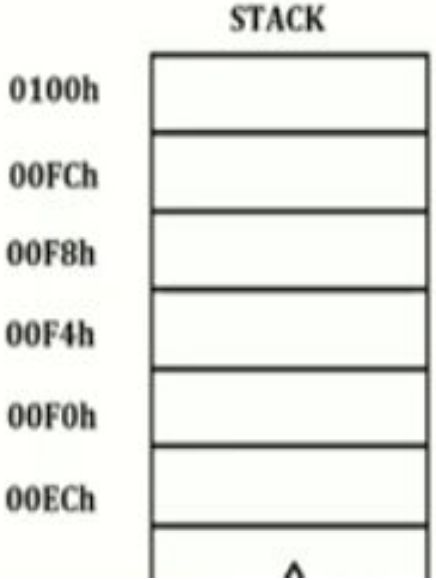
Lecture 10 Procedures Part 2

Example to Understand Stack & Procedure

Write down the contents of EAX,EBX & ECX by debugging this Code?
Also update the Stack step by step according to given code?

```
1.  .Model Small
2.  .Stack 100h
3.  .Data
4.  .Code
5.  Main PROC
6.  PUSH 5
7.  PUSH 7
8.  CALL SUMOF
9.  POP EBX
10. POP ECX
11. Main endp
```

```
i.    SUMOF PROC
ii.   PUSH EBP
iii.  Mov EBP,ESP
iv.   MOV EAX ,0
v.    Add EAX, [EBP+8]
vi.   Add EAX ,[EBP+12]
vii.  POP EBP
viii. RET
ix.   SUMOF ENDP
12.   End Main
```



Recursion in Assembly Language

```
.stack 0100h
.data
.code
Main PROC
Mov ecx , 5
Mov eax , 0
Call CalcSum
Exit
Main Endp
```

```
CalcSum PROC
Cmp ecx, 0
Jz L2
Add eax,ecx
Dec ecx
Call CalcSum
L2 : RET
CalcSum Endp
End main
```

Question:

Fill in the Stack while doing dry run. Also tell the final Contents of EAX and ECX. In case of any Error(s), specify clearly. What is the purpose of this code?

	Top of Stack	ECX	EAX
00FCh			
00F8h			
00F4h			
00F0h			
00ECh			
00E8h			